Docker Fundamentals

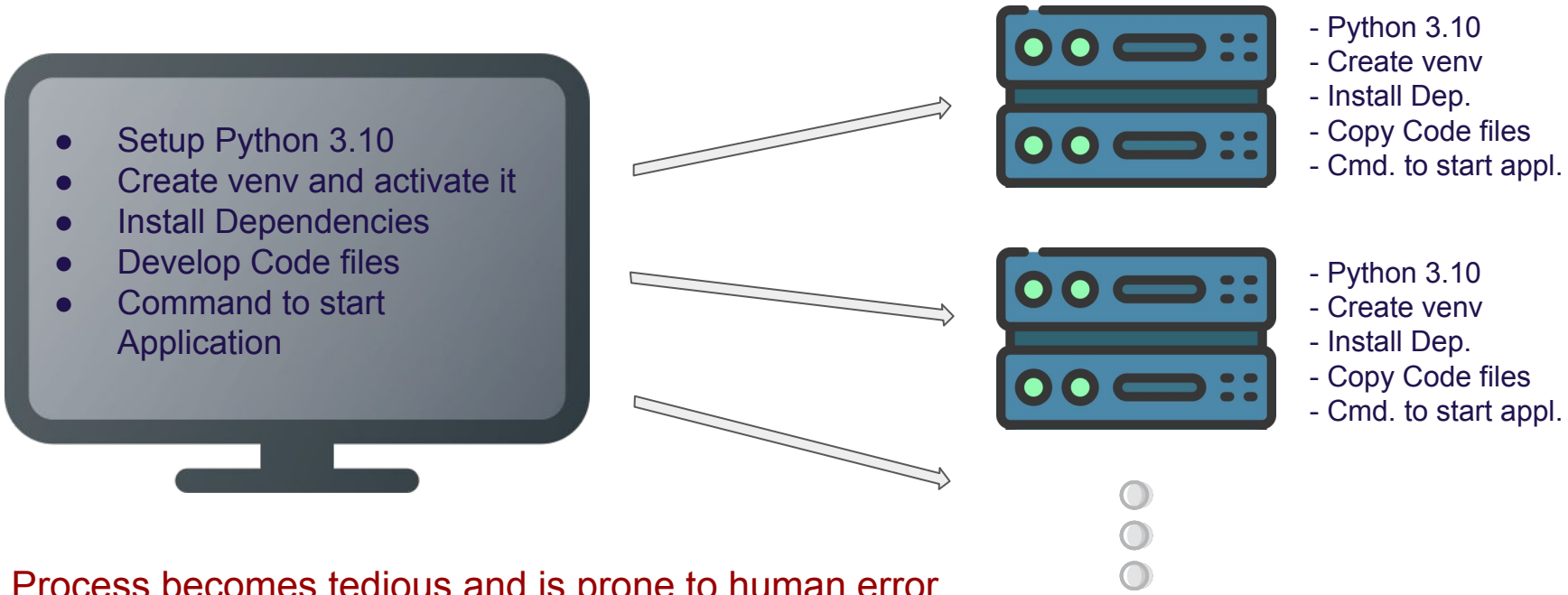
Containers and Images

Objectives

1. Why Docker? (The Problems Before Containerization)
2. Introduction to Docker
3. Containerize your Application
4. Docker Images and Containers
5. Basic Docker Commands
6. Hands-on: Docker Basics

Why Docker? (The Problems Before Containerization)

How to Run your Application on a Server? - First, Build your application on your system, then take it to server



Process becomes tedious and is prone to human error

Why Docker? (..contd)

Challenges Faced by Developers:

Developers had to spend time to ensure they have the right setup and configuration for every new environment.

- **Environment Mismatch**:
  - "Works on my machine" issues.
  - Variations in OS, libraries, and dependencies.
- **Resource Overhead**:
  - Traditional Virtual Machines (VMs) were resource-intensive.
  - Slow startup times.
- **Scalability Issues**:
  - Difficulty in efficiently scaling applications.
- **Complex Deployment**:
  - Managing dependencies and versions across environments.

Solution: Containerization:

- Lightweight, portable, and isolated environments.

Introduction to Docker

What is Docker?

- A platform for building, sharing, and running applications in containers.

Key Components:

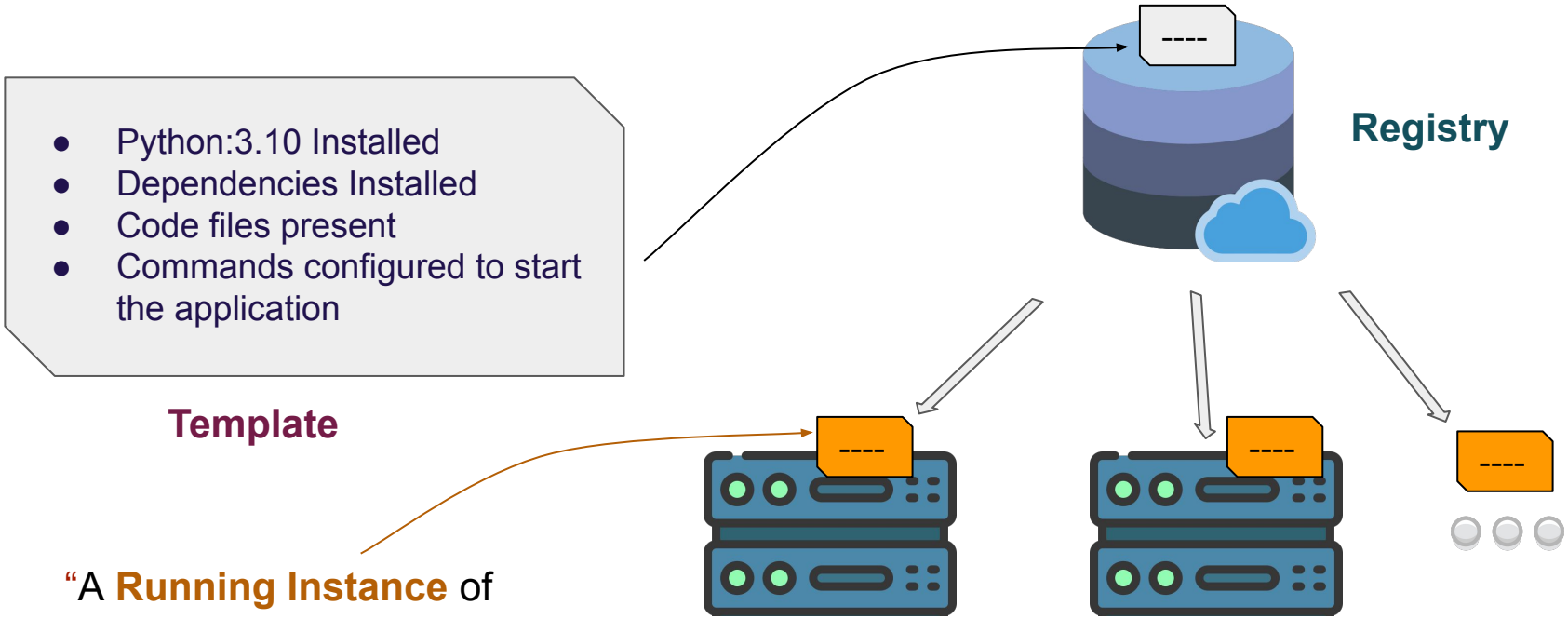
- **Images**: Lightweight, standalone, and executable software packages.
- **Containers**: Runtime instances of images.

Why Use Docker?

- Portability: "Write once, run anywhere."
- Efficiency: Lightweight compared to virtual machines.

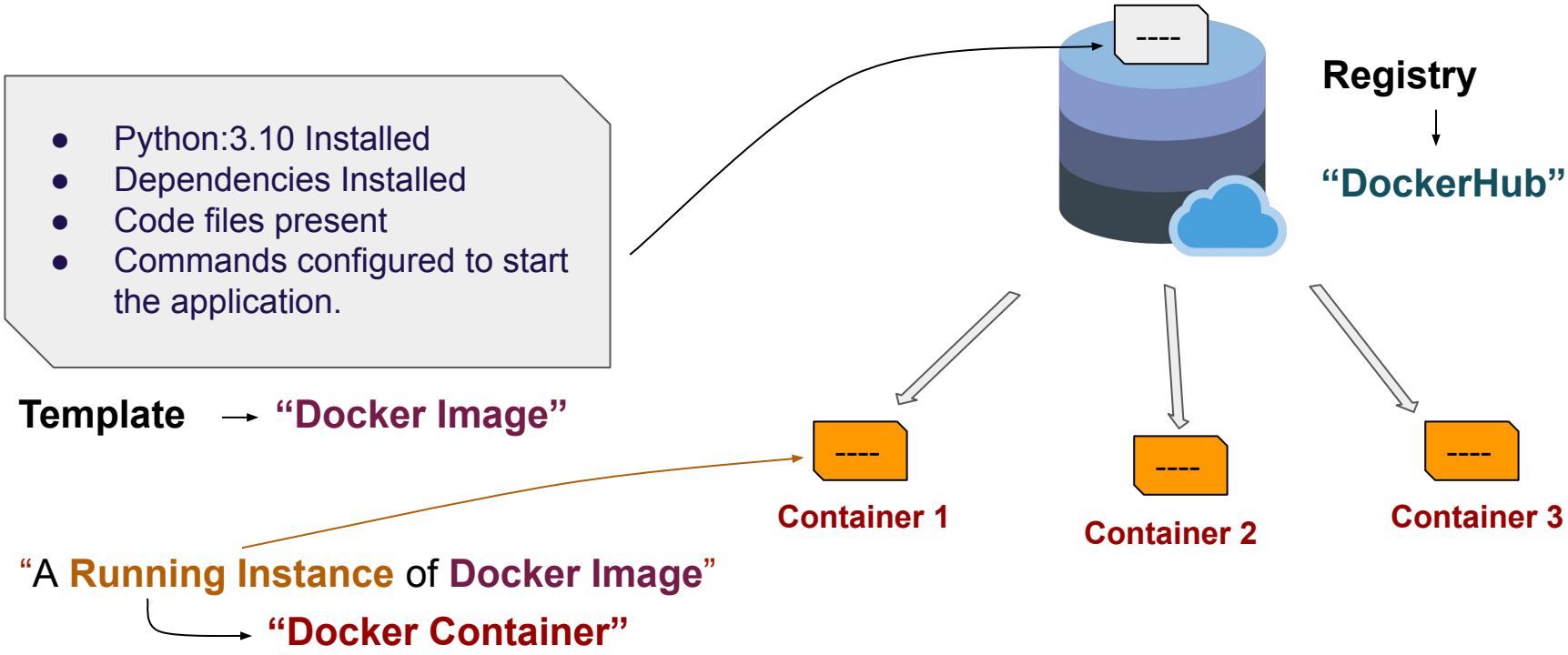
Containerize your Application

- Once the application is built on your system, create a **template** of it
- Use **an instance** of that **template** to run the app anywhere



Containerize Application (with Docker)

- Few terminologies to note:



Understanding Docker Images

What are Docker Images?

- Templates for creating containers.
- Composed of layers, each representing changes or instructions.

Dockerfile:

- A script used to build an image.
- Key commands: FROM, RUN, COPY, CMD.

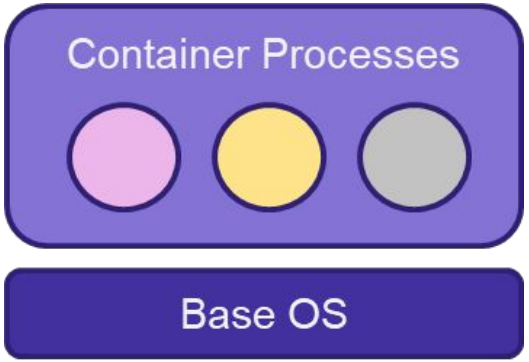
Registry:

- Stores and shares images (e.g., Docker Hub, private registries).

Understanding Docker Containers

What is a Docker Container?

- A lightweight, isolated environment to run applications.
- Created from Docker images.



Key Properties:

- Isolated filesystem, networking, and processes.
- Ephemeral: Changes vanish unless saved as a new image.

Common Commands:

- `docker run`: Start a container.
- `docker ps`: List running containers.
- `docker stop`: Stop a container.

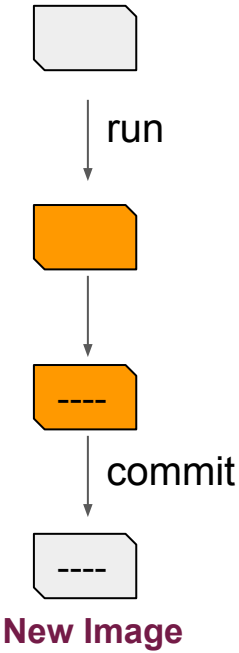
Docker Images vs. Containers

Feature	Docker Image	Docker Container
Definition	Blueprint for containers	Running instance of an image
Storage	Static and reusable	Dynamic, reflects runtime changes
Purpose	Build and distribute containers	Execute applications

How to create a “Docker Image”?

- Without **Dockerfile**

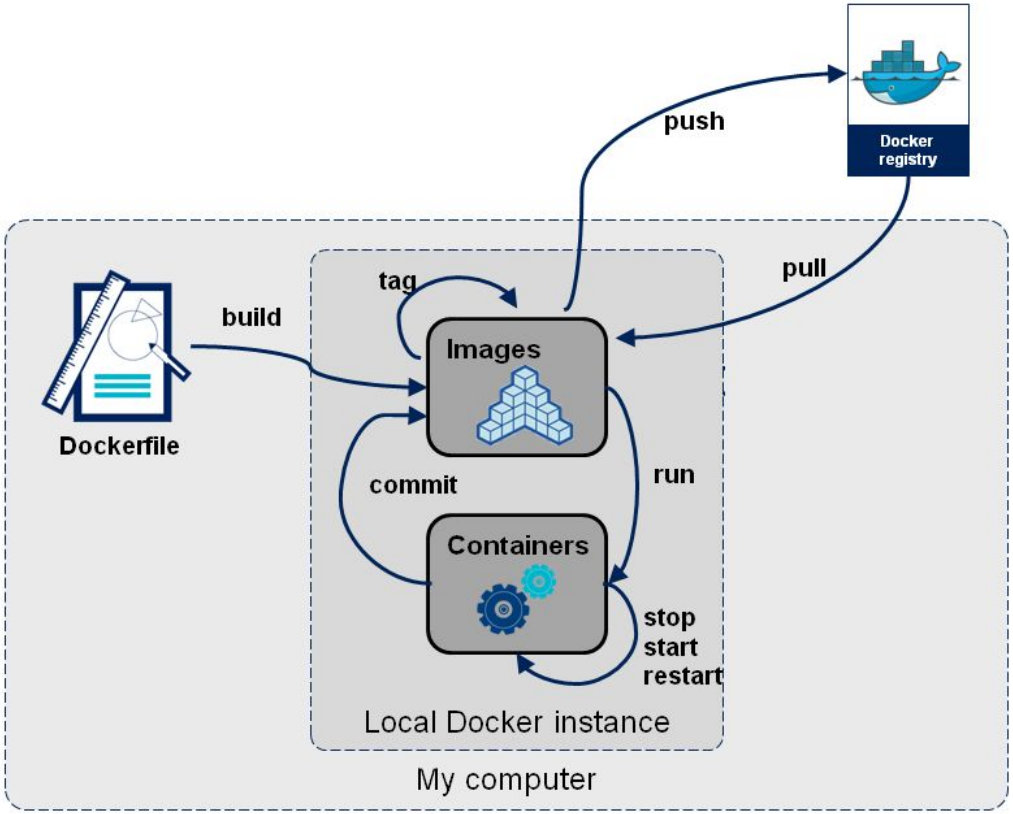
1. Run a Container using a **Base Image**
2. Inside Container
   a. **Manually** add your files
   b. Install dependencies
3. **Commit** the changes



- With **Dockerfile**

```
# pull python base image
FROM python:3.10
# copy application files
ADD . .
# specify working directory
WORKDIR /titanic_model_api
# update pip
RUN pip install --upgrade pip
# install dependencies
RUN pip install -r requirements.txt
# expose port for application
EXPOSE 8001
# start fastapi application
CMD ["python", "app/main.py"]
```

Basic Docker Commands




```
>> docker build . -t imgName:1
>> docker run --name=myApp imgName:1
        >> docker stop myApp
        >> docker start myApp
        >> docker restart myApp
        >> docker commit myApp newImg:1
>> docker tag newImg:1 username/newImg:2
>> docker push username/newImg:2
>> docker pull username/newImg:2
```

Docker Setup and Basics

Refer to Docker Setup and Basics.pdf

Demo...


Thank you!